

编译原理 第十次作业

姓名：朱首赫

学号：2023K8009906029

1 练习 6.2.1

假定图 6-26 中的函数 widen 可以处理图 6-25a 的层次结构中的所有类型，翻译下列表达式。假定 c 是 char 型，i 是 int 型，s 是 short 型，f 是 float 型，y 是 double 型。

1. $f = i * c$
2. $y = (i + f) - (s + c)$
1. 翻译为三地址代码：

$$\begin{aligned}t_1 &= (\text{int})c \\t_2 &= i * t_1 \\t_3 &= (\text{float})t_2 \\f &= t_3\end{aligned}$$

2. 翻译为三地址代码：

$$\begin{aligned}t_1 &= (\text{float})i \\t_2 &= t_1 + f \\t_3 &= (\text{short})c \\t_4 &= s + t_3 \\t_5 &= (\text{float})t_4 \\t_6 &= t_2 - t_5 \\t_7 &= (\text{double})t_6 \\y &= t_7\end{aligned}$$

2 练习 6.2.2

在图 6-36 的语法制导定义中添加处理下列控制流构造的规则：

1. $S \rightarrow \text{repeat } S1 \text{ until } B$ ，当 B 为真时结束循环
2. $S \rightarrow \text{for } (S1 ; B ; S2) S3$

表 1: 新增控制流构造的语法制导定义

产生式	语义规则
$S \rightarrow \text{repeat } S_1 \text{ until } B$	$S_1.\text{next} = \text{newlabel}()$ $B.\text{false} = \text{newlabel}()$ $B.\text{true} = S.\text{next}$ $S.\text{code} = \text{label}(B.\text{false}) \parallel S_1.\text{code} \parallel \text{label}(S_1.\text{next}) \parallel B.\text{code}$
$S \rightarrow \text{for } (S_1; B; S_2) S_3$	$S_1.\text{next} = \text{newlabel}()$ $B.\text{true} = \text{newlabel}()$ $B.\text{false} = S.\text{next}$ $S_3.\text{next} = \text{newlabel}()$ $S_2.\text{next} = S_1.\text{next}$ $S.\text{code} = S_1.\text{code} \parallel \text{label}(S_1.\text{next}) \parallel B.\text{code}$ $\parallel \text{label}(B.\text{true}) \parallel S_3.\text{code}$ $\parallel \text{label}(S_3.\text{next}) \parallel S_2.\text{code}$ $\parallel \text{gen}(\text{'goto' } S_1.\text{next})$

3 练习 6.2.3

使用图 6-43 中的翻译方案翻译下列表达式。给出每个子表达式的 truelist 和 falselist。你可以假设第一条被生成的指令的地址是 100。

1. $a == b \ \&\& \ (c == d \ || \ e == f)$
2. $(a == b \ || \ c == d) \ || \ e == f$
3. $(a == b \ \&\& \ c == d) \ \&\& \ e == f$

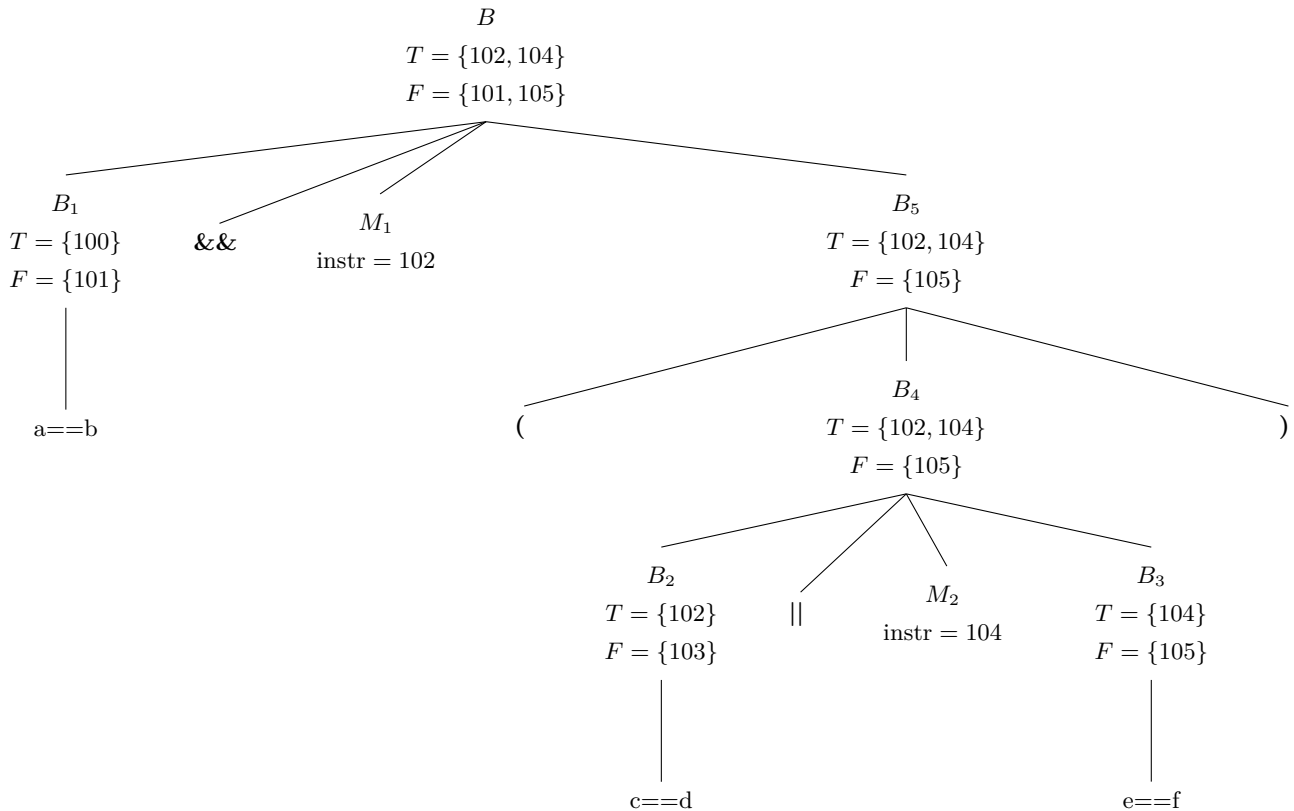
1. 表达式: $a == b \ \&\& \ (c == d \ || \ e == f)$

分析过程与属性计算

- $B_1 \rightarrow a == b$
 - 生成指令: 100: if $a == b$ goto $_$, 101: goto $_$
 - $B_1.\text{truelist} = \{100\}$, $B_1.\text{falselist} = \{101\}$
- $M_1 \rightarrow \epsilon$
 - $M_1.\text{instr} = 102$
 - 触发 $B \rightarrow B_1 \ \&\& \ M_1 \ B_5$ 中的 $\text{backpatch}(B_1.\text{truelist}, M_1.\text{instr})$: 将 100 处的 goto $_$ 填为 goto 102。
- $B_2 \rightarrow c == d$
 - 生成指令: 102: if $c == d$ goto $_$, 103: goto $_$

- $B_2.truelist = \{102\}, B_2.falselist = \{103\}$
- $M_2 \rightarrow \epsilon$
 - $M_2.instr = 104$
 - 触发 $B_4 \rightarrow B_2 \parallel M_2 B_3$ 中的 $backpatch(B_2.falselist, M_2.instr)$: 将 103 处的 `goto _` 填为 `goto 104`。
- $B_3 \rightarrow e == f$
 - 生成指令: 104: `if e==f goto _`, 105: `goto _`
 - $B_3.truelist = \{104\}, B_3.falselist = \{105\}$
- $B_4 \rightarrow B_2 \parallel M_2 B_3$
 - $B_4.truelist = merge(B_2.truelist, B_3.truelist) = \{102, 104\}$
 - $B_4.falselist = B_3.falselist = \{105\}$
- $B_5 \rightarrow (B_4)$
 - $B_5.truelist = B_4.truelist = \{102, 104\}$
 - $B_5.falselist = B_4.falselist = \{105\}$
- $B \rightarrow B_1 \ \&\& \ M_1 \ B_5$
 - $B.truelist = B_5.truelist = \{102, 104\}$
 - $B.falselist = merge(B_1.falselist, B_5.falselist) = \{101, 105\}$

带注释的语法分析树

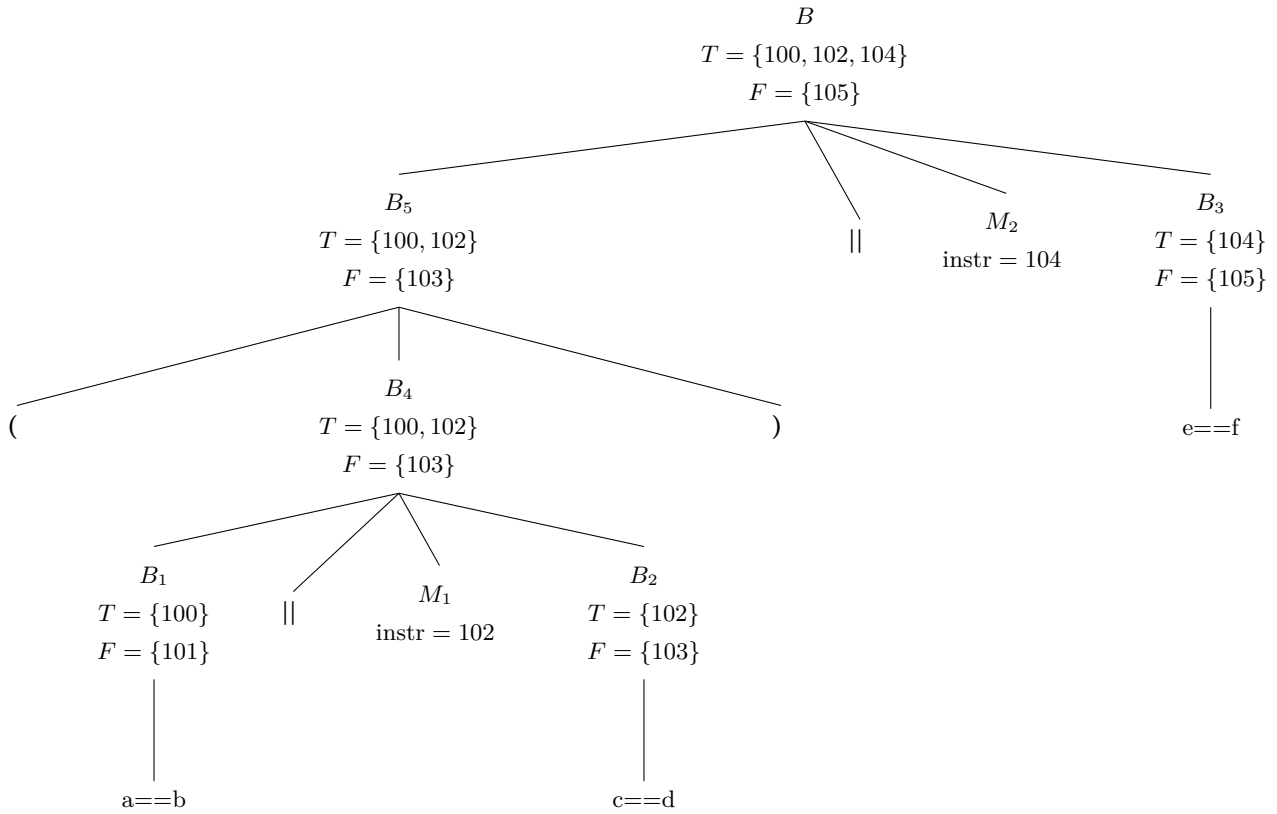


2. 表达式: $(a==b \ || \ c==d) \ || \ e==f$

分析过程与属性计算

- $B_1 \rightarrow a == b$
 - 生成指令: 100: if a==b goto _, 101: goto _
 - $B_1.truelist = \{100\}$, $B_1.falselist = \{101\}$
- $M_1 \rightarrow \epsilon$
 - $M_1.instr = 102$
 - 触发 `backpatch( $B_1.falselist, M_1.instr$ ): 101 填为 goto 102。`
- $B_2 \rightarrow c == d$
 - 生成指令: 102: if c==d goto _, 103: goto _
 - $B_2.truelist = \{102\}$, $B_2.falselist = \{103\}$
- $B_4 \rightarrow B_1 \ || \ M_1 \ B_2$
 - $B_4.truelist = merge(\{100\}, \{102\}) = \{100, 102\}$
 - $B_4.falselist = \{103\}$
- $B_5 \rightarrow (B_4)$
 - $B_5.truelist = \{100, 102\}$
 - $B_5.falselist = \{103\}$
- $M_2 \rightarrow \epsilon$
 - $M_2.instr = 104$
 - 触发 `backpatch( $B_5.falselist, M_2.instr$ ): 103 填为 goto 104。`
- $B_3 \rightarrow e == f$
 - 生成指令: 104: if e==f goto _, 105: goto _
 - $B_3.truelist = \{104\}$, $B_3.falselist = \{105\}$
- $B \rightarrow B_5 \ || \ M_2 \ B_3$
 - $B.truelist = merge(B_5.truelist, B_3.truelist) = \{100, 102, 104\}$
 - $B.falselist = B_3.falselist = \{105\}$

带注释的语法分析树



3. 表达式: $(a == b \ \&\& \ c == d) \ \&\& \ e == f$

分析过程与属性计算

- $B_1 \rightarrow a == b$
 - 生成指令: 100: if a==b goto _, 101: goto _
 - $B_1.truelist = \{100\}$, $B_1.falselist = \{101\}$
- $M_1 \rightarrow \epsilon$
 - $M_1.instr = 102$
 - 触发 `backpatch( $B_1.truelist, M_1.instr$ )`: 100 填为 goto 102。
- $B_2 \rightarrow c == d$
 - 生成指令: 102: if c==d goto _, 103: goto _
 - $B_2.truelist = \{102\}$, $B_2.falselist = \{103\}$
- $B_4 \rightarrow B_1 \ \&\& \ M_1 \ B_2$
 - $B_4.truelist = B_2.truelist = \{102\}$
 - $B_4.falselist = merge(B_1.falselist, B_2.falselist) = \{101, 103\}$
- $B_5 \rightarrow (B_4)$

- $B_5.truelist = \{102\}$
- $B_5.falselist = \{101, 103\}$
- $M_2 \rightarrow \epsilon$
 - $M_2.instr = 104$
 - 触发 `backpatch( $B_5.truelist, M_2.instr$ ): 102 填为 goto 104。`
- $B_3 \rightarrow e == f$
 - 生成指令: 104: if e==f goto _, 105: goto _
 - $B_3.truelist = \{104\}, B_3.falselist = \{105\}$
- $B \rightarrow B_5 \ \&\& \ M_2 \ B_3$
 - $B.truelist = B_3.truelist = \{104\}$
 - $B.falselist = merge(B_5.falselist, B_3.falselist) = \{101, 103, 105\}$

带注释的语法分析树

